

实验报告

课程:机器学习	实验名称:集成学习	日期:5.18
姓名:雷文豪	学号:138022024031	专业班级:人工智能

1 实验目的

1. 掌握集成学习的原理，使用集成学习方法求解问题。
2. 学习解题设计、通过代码编写与运行实施解题、完成程序正确性的验证。

2 实验内容提要 with 实验设备

1. 给待解问题设计求解方法和集成学习模型。
2. 对给定数据集，设计模型的求解方法、编写程序实施求解。
3. 实验结果的分析与确认。
4. 实验设备：个人电脑或便携式电脑，并安装 Windows 等操作系统和编程开发环境。

3 实验内容与步骤

3.1 二分类问题（模仿实例例题求解）

参考课本的实例例题，对乳腺癌（breast cancer）数据集，采用并行方式和串行方式的集成学习方法，分别建立分类模型判别样本的类别，并评估所设计的分类模型的预测性能。

- 简单写出你的求解设计（例如：如何处理原始数据、选择何种分类方法、所选分类方法的简单原理、如何实行平均泛化性能的评估）。
- 获得的结果，即模型的泛化性能。
- 程序运行成功的截图（包含源文件名 < 字母 + 学号后 3 位>、运行光标在程序尾行）。

（一）求解设计

- **数据预处理：**加载乳腺癌数据集，使用StandardScaler进行标准化处理，按7:3比例划分训练集和测试集（分层采样保持类别比例）。
- **模型选择与原理：**
 - **Bagging（并行集成）：**以决策树为基学习器，通过Bootstrap自助采样生成多个训练子集，并行训练多个弱学习器，最终通过投票法得到分类结果。n_estimators=50, max_depth=5。
 - **随机森林（并行集成）：**Bagging的改进版本，在Bagging基础上引入随机特征选择，进一步降低方差。n_estimators=100。
 - **AdaBoost（串行集成）：**以决策树为基学习器，通过迭代调整样本权重，每次训练聚焦于前一轮分类错误的样本，最后加权组合所有弱分类器。n_estimators=100, learning_rate=0.8。
 - **GBDT（串行集成）：**梯度提升决策树，每轮训练拟合前一轮的残差（负梯度），通过逐步减少残差来提升模型性能。n_estimators=100, learning_rate=0.1。
- **性能评估：**使用10折交叉验证评估平均泛化性能，输出准确率(Accuracy)、精确率(Precision)、召回率(Recall)、F1分数。
- **参数影响探究：**考察不同树深度对随机森林性能的影响，以及不同基学习器数量对AdaBoost性能的影响。
- **核心结果：**
 - 所有集成方法在测试集上准确率均超过94%
 - AdaBoost表现最优：测试准确率97.08%，10折CV准确率97.01%

（二）结果分析

➤ 运行结果

=====
任务3.1：乳腺癌数据集二分类 - 集成学习方法
=====

数据集大小：569 个样本，30 个特征

类别分布：[212 357] (0=恶性, 1=良性)

特征名称：[np.str_('mean radius'), np.str_('mean texture'), np.str_('mean perimeter'), np.str_('mean area'), np.str_('mean smoothness'), np.str_('mean compactness'), np.str_('mean concavity'), np.str_('mean concave points'), np.str_('mean symmetry'), np.str_('mean fractal dimension'), np.str_('radius error'), np.str_('texture error'), np.str_('perimeter error'), np.str_('area error'), np.str_('smoothness error'), np.str_('compactness error'), np.str_('concavity error'), np.str_('concave points error'), np.str_('symmetry error'), np.str_('fractal dimension error'), np.str_('worst radius'), np.str_('worst texture'), np.str_('worst perimeter'), np.str_('worst area'), np.str_('worst smoothness'), np.str_('worst compactness'), np.str_('worst concavity'), np.str_('worst concave points'), np.str_('worst symmetry'), np.str_('worst fractal dimension')]

训练集大小：398，测试集大小：171

=====
模型训练与评估
=====

- [1] Bagging (并行集成) – 基学习器: 决策树
- [2] 随机森林 (并行集成)
- [3] AdaBoost (串行集成) – 基学习器: 决策树
- [4] GBDT (串行集成)

=====

模型性能评估结果

=====

Bagging:

准确率 (Accuracy): 0.9415
精确率 (Precision): 0.9450
召回率 (Recall): 0.9626
F1分数: 0.9537

随机森林:

准确率 (Accuracy): 0.9415
精确率 (Precision): 0.9450
召回率 (Recall): 0.9626
F1分数: 0.9537

AdaBoost:

准确率 (Accuracy): 0.9708
精确率 (Precision): 0.9554
召回率 (Recall): 1.0000
F1分数: 0.9772

GBDT:

准确率 (Accuracy): 0.9532
精确率 (Precision): 0.9459
召回率 (Recall): 0.9813
F1分数: 0.9633

=====

十折交叉验证 (评估平均泛化性能)

=====

Bagging:

10折交叉验证准确率: 0.9544 (+/- 0.0612)

随机森林:

10折交叉验证准确率: 0.9632 (+/- 0.0776)

AdaBoost:

10折交叉验证准确率: 0.9701 (+/- 0.0353)

GBDT:

10折交叉验证准确率: 0.9649 (+/- 0.0544)

参数影响探究: 不同树深度对随机森林性能的影响

最大深度 = 3: 测试准确率 = 0.9357
最大深度 = 5: 测试准确率 = 0.9415
最大深度 = 8: 测试准确率 = 0.9357
最大深度 = 10: 测试准确率 = 0.9357
最大深度 = 15: 测试准确率 = 0.9357
最大深度 = 无限制: 测试准确率 = 0.9357

不同基学习器数量对AdaBoost性能的影响

基学习器数量 = 10: 测试准确率 = 0.9591
基学习器数量 = 30: 测试准确率 = 0.9474
基学习器数量 = 50: 测试准确率 = 0.9649
基学习器数量 = 100: 测试准确率 = 0.9708
基学习器数量 = 200: 测试准确率 = 0.9766

➤ 分析

1. 总体表现

所有四种集成学习方法 (Bagging、随机森林、AdaBoost、GBDT) 在乳腺癌数据集上都表现出了优秀的分类能力, 测试准确率均超过94%。这说明对于该数据集, 基于决策树的集成模型是非常有效的选择。值得注意的是, 数据集中存在轻微类别不平衡 (良性357例, 恶性212例), 比例约为1.7:1, 模型整体对多数类可能存在轻微偏好。

2. 各模型性能对比与解读

2.1 AdaBoost表现最佳

AdaBoost在所有模型中取得了最高的测试准确率 (0.9708) 和最高的F1分数 (0.9772)。最关键的是, 其召回率达到1.0000, 这意味着在测试集上, 所有恶性样本都被正确识别, 没有漏诊。在乳腺癌诊断场景中, 这个指标至关重要, 因为漏诊的代价远高于误诊。十折交叉验证中, AdaBoost不仅均分最高 (0.9701), 而且标准差最小 (± 0.0353), 说明其泛化性能最稳定, 受数据划分方式的影响较小。

2.2 Bagging与随机森林表现一致

Bagging和随机森林的四个指标完全相同 (准确率0.9415, F1分数0.9537)。理论上随机森林在Bagging的基础上引入了特征随机选择, 通常能带来更多多样性从而提升性能, 但在此数据集上未见差异, 可能原因是该数据集特征之间冗余度较低, 或者基学习器数量已足够多, 使得两种方法的方差缩减效果趋同。

2.3 GBDT表现中规中矩

GBDT的准确率 (0.9532) 和F1分数 (0.9633) 介于AdaBoost和随机森林之间, 表现良好但不如AdaBoost突出。这可能因为对于这个规模适中的数据, Boosting策略 (串行修正错误) 比Bagging策略 (并行平均) 更能聚焦于难分类样本, 而AdaBoost对错误分类样本的权重调整机制在此数据集上尤为有效。

3. 参数影响分析

3.1 随机森林的树深度影响

当最大深度从3增加到5时，准确率从0.9357提升到0.9415，但继续增加深度（8、10、15甚至无限制）后，准确率反而下降并稳定在0.9357。这说明：树深度为5时模型已达到最优复杂度；继续增加深度导致过拟合，模型开始学习训练数据中的噪声，泛化能力下降。深度无限制时，决策树会生长到每个叶子节点纯净为止，过拟合风险最大。

3.2 AdaBoost的基学习器数量影响

随着基学习器数量从10增加到200，测试准确率整体呈上升趋势（从0.9591到0.9766）。这表明更多的弱学习器能够逐步修正更多错误，持续提升性能。值得注意的是，基学习器数量为30时准确率（0.9474）反而低于10个时的准确率，这可能是因为数据划分的随机波动造成，也可能是该数量下模型暂时陷入了局部状态。总体趋势仍然是基学习器越多，性能越好，但边际收益递减——从50个到200个，准确率提升从0.9649到0.9766，增幅逐渐放缓。

4. 结论与建议

综合来看，AdaBoost是该乳腺癌数据集上的最优模型，兼顾了高准确率、完美召回率和稳定泛化能力，最符合医学诊断中对“不漏诊”的严格要求。随机森林和Bagging是稳定且解释性强的基线选择，GBDT也提供了优秀的性能。对于类似的中小规模表格数据集，Boosting类方法（尤其是AdaBoost）值得优先考虑。同时，在实际应用中需要合理控制模型复杂度（如限制树深度），防止过拟合，以确保模型在真实新样本上能保持优秀的预测能力。

（三） 截图

```
task3_1_classification.py X
D:\Studies\机器学习> src> ch10> task3_1_classification.py -
154 n_estimators=n_est, learning_rate=0.8, random_state=42
155 )
156 ada_temp.fit(X_train, y_train)
157 y_pred_temp = ada_temp.predict(X_test)
158 acc = accuracy_score(y_test, y_pred_temp)
159 print(f" 基学习器数量 = {n_est}; 测试准确率 = {acc:.4f}")
160
161 print("\n" + "-" * 60)
162 print("任务3.1 程序运行完成!")
163 print("-" * 60)

(machine_learning_env) PS D:\Visual Studio Code\Microsoft VS Code> & D:\miniconda\envs\machine_learning_env\python.exe d:\Studies\机器学习\src\ch10\task3_1_classification.py

十折交叉验证 (评估平均泛化性能)

=====
Bagging:
10折交叉验证准确率: 0.9544 (+/- 0.0012)

随机森林:
10折交叉验证准确率: 0.9632 (+/- 0.0776)

AdaBoost:
10折交叉验证准确率: 0.9701 (+/- 0.0353)

GBDT:
10折交叉验证准确率: 0.9649 (+/- 0.0544)

=====
参数影响探究: 不同树深度对随机森林性能的影响
最大深度 = 3: 测试准确率 = 0.9357
最大深度 = 5: 测试准确率 = 0.9415
最大深度 = 8: 测试准确率 = 0.9357
最大深度 = 10: 测试准确率 = 0.9357
最大深度 = 15: 测试准确率 = 0.9357
最大深度 = 无限制: 测试准确率 = 0.9357

=====
不同基学习器数量对AdaBoost性能的影响
基学习器数量 = 10: 测试准确率 = 0.9591
基学习器数量 = 30: 测试准确率 = 0.9474
基学习器数量 = 50: 测试准确率 = 0.9649
基学习器数量 = 100: 测试准确率 = 0.9708
基学习器数量 = 200: 测试准确率 = 0.9766

=====
任务3.1 程序运行完成!

(machine_learning_env) PS D:\Visual Studio Code\Microsoft VS Code>
```

3.2 回归问题（模仿实例例题求解）

参考课本的实例例题，采用集成学习的回归方法，对体脂（bodyfat）数据集进行分析，并评估回归模型的回归效果。

- 简单写出你的求解设计（例如：如何处理原始数据、选择何种集成学习的回归方法、所选方法的简单原理、如何实行平均泛化性能的评估）。
- 获得的结果，即模型的泛化性能。
- 程序运行成功的截图（包含源文件名 < 字母 + 学号后 3 位>、运行光标在程序尾行）。

（一）求解设计

➤ **数据预处理：**读取bodyfat.txt（空格分隔），添加列名，提取13个身体测量指标作为特征，体脂率(BodyFat)作为目标变量，标准化处理后按7:3划分训练/测试集。

➤ **模型选择与原理：**

- **决策树回归：**以树结构递归划分特征空间，max_depth=5。
- **Bagging回归：**并行训练多个决策树回归器，平均预测结果降低方差。
- **随机森林回归：**Bagging + 随机特征选择，n_estimators=100。
- **AdaBoost回归：**串行提升，关注预测误差大的样本。
- **GBDT回归：**梯度提升回归，逐步拟合残差，n_estimators=100, learning_rate=0.1。

➤ **性能评估：**使用MSE、MAE、 R^2 三个指标评估回归效果，并进行10折交叉验证。

➤ **特征重要性分析：**使用随机森林输出各特征对体脂率预测的贡献度排名。

➤ **核心结果：**

- 集成方法显著优于单棵决策树（测试 R^2 从0.45提升至0.67）
- AdaBoost回归在测试集上表现最佳（ $R^2=0.6673$ ）
- 腹围(Abdomen)是最重要的预测特征（重要性0.7433）

（二）结果分析

➤ **运行结果**

=====

任务3.2: 体脂(bodyfat)数据集回归 - 集成学习方法

=====

数据集大小: 252 个样本, 15 个特征

数据前5行:

	Density	BodyFat	Age	Weight	Height	Neck	Chest	Abdomen	Hip	Thigh	Knee	Ankle	Biceps	Forearm	Wrist
0	1.0708	12.3	23	154.25	67.75	36.2	93.1	85.2	94.5	59.0	37.3	21.9	32.0	27.4	17.1
1	1.0853	6.1	22	173.25	72.25	38.5	93.6	83.0	98.7	58.7	37.3	23.4	30.5	28.9	18.2
2	1.0414	25.3	22	154.00	66.25	34.0	95.8	87.9	99.2	59.6	38.9	24.0	28.8	25.2	16.6
3	1.0751	10.4	26	184.75	72.25	37.4	101.8	86.4	101.2	60.1	37.3	22.8	32.4	29.4	18.2
4	1.0340	28.7	24	184.25	71.25	34.4	97.3	100.0	101.9	63.2	42.2	24.0	32.2	27.7	17.7

数据统计描述:

	Density	BodyFat	Age	Weight	Height	Neck	Chest	Abdomen	Hip	Thigh	Knee	Ankle	Biceps	Forearm	Wrist
count	252.000000	252.000000	252.000000	252.000000	252.000000	252.000000	252.000000	252.000000	252.000000	252.000000	252.000000	252.000000	252.000000	252.000000	252.000000
mean	1.055574	19.150794	44.884921	178.924405	70.148810	37.992063	100.824206	92.555952	99.904762	59.405952	38.590476	23.102381	32.273413	28.663889	18.229762
std	0.019031	8.368740	12.602040	29.389160	3.662856	2.430913	8.430476	10.783077	7.164058	5.249952	2.411805	1.694893	3.021274	2.020691	0.933585
min	0.995000	0.000000	22.000000	118.500000	29.500000	31.100000	79.300000	69.400000	85.000000	47.200000	33.000000	19.100000	24.800000	21.000000	15.800000
25%	1.041400	12.475000	35.750000	159.000000	68.250000	36.400000	94.350000	84.575000	95.500000	56.000000	36.975000	22.000000	30.200000	27.300000	17.600000
50%	1.054900	19.200000	43.000000	176.500000	70.000000	38.000000	99.650000	90.950000	99.300000	59.000000	38.500000	22.800000	32.050000	28.700000	18.300000
75%	1.070400	25.300000	54.000000	197.000000	72.250000	39.425000	105.375000	99.325000	103.525000	62.350000	39.925000	24.000000	34.325000	30.000000	18.800000
max	1.108900	47.500000	81.000000	363.150000	77.750000	51.200000	136.200000	148.100000	147.700000	87.300000	49.100000	33.900000	45.000000	34.900000	21.400000

特征数量: 13

特征列: ['Age', 'Weight', 'Height', 'Neck', 'Chest', 'Abdomen', 'Hip', 'Thigh', 'Knee', 'Ankle', 'Biceps', 'Forearm', 'Wrist']

目标变量 BodyFat 统计: 均值=19.15, 标准差=8.37, 最小值=0.0, 最大值=47.5

缺失值数量: 0

训练集大小: 176, 测试集大小: 76

集成学习回归模型训练与评估

决策树回归:

训练集 - MSE: 8.0440, MAE: 2.0803, R^2 : 0.8955

测试集 - MSE: 28.4731, MAE: 4.2914, R^2 : 0.4499

Bagging回归:

训练集 - MSE: 5.0888, MAE: 1.8081, R^2 : 0.9339

测试集 - MSE: 17.7420, MAE: 3.4969, R^2 : 0.6572

随机森林回归:

训练集 - MSE: 5.2152, MAE: 1.8003, R^2 : 0.9322

测试集 - MSE: 18.2289, MAE: 3.5361, R^2 : 0.6478

AdaBoost回归:

训练集 - MSE: 6.4907, MAE: 2.2593, R^2 : 0.9156

测试集 - MSE: 17.2204, MAE: 3.3693, R^2 : 0.6673

GBDT回归:

训练集 - MSE: 0.2503, MAE: 0.4171, R^2 : 0.9967

测试集 - MSE: 20.6390, MAE: 3.8419, R^2 : 0.6013

模型性能汇总表

模型	训练MSE	测试MSE	训练MAE	测试MAE	训练 R^2	测试 R^2
决策树回归	8.0440	28.4731	2.0803	4.2914	0.8955	0.4499
Bagging回归	5.0888	17.7420	1.8081	3.4969	0.9339	0.6572
随机森林回归	5.2152	18.2289	1.8003	3.5361	0.9322	0.6478
AdaBoost回归	6.4907	17.2204	2.2593	3.3693	0.9156	0.6673
GBDT回归	0.2503	20.6390	0.4171	3.8419	0.9967	0.6013

最佳性能模型: AdaBoost回归 (测试 R^2 = 0.6673)

=====

十折交叉验证 (评估平均泛化性能)

=====

决策树回归:

10折交叉验证 R^2 : 0.3413 (+/- 0.5402)

Bagging回归:

10折交叉验证 R^2 : 0.5520 (+/- 0.5116)

随机森林回归:

10折交叉验证 R^2 : 0.5452 (+/- 0.4910)

AdaBoost回归:

10折交叉验证 R^2 : 0.5571 (+/- 0.4890)

GBDT回归:

10折交叉验证 R^2 : 0.5190 (+/- 0.6283)

=====

特征重要性分析 (随机森林)

=====

特征重要性排名:

1. Abdomen: 0.7433
2. Height: 0.0439
3. Age: 0.0294
4. Thigh: 0.0276
5. Weight: 0.0233
6. Hip: 0.0232
7. Wrist: 0.0189
8. Chest: 0.0188
9. Knee: 0.0159
10. Ankle: 0.0158
11. Neck: 0.0135
12. Forearm: 0.0132
13. Biceps: 0.0131

➤ 分析

1. 总体表现概述

本任务使用集成学习方法对体脂率进行回归预测。整体来看，所有模型在训练集上表现良好甚至出现过拟合倾向，但在测试集上的泛化能力有限，最佳模型AdaBoost的测试 R^2 仅为0.6673。这表明体脂率预测任务相对于此前的分类任务更具挑战性，原因可能在于：样本量较小（仅252个），特征与目标变量之间的关系可能并非严格的线性或树结构所能完全捕捉，以及数据中存在一定噪声。

2. 关键发现：严重过拟合问题

本次实验最显著的现象是多个模型出现了训练集与测试集性能的严重背离：

2.1 决策树回归：训练 $R^2=0.8955$ ，测试 $R^2=0.4499$ ，差值高达0.4456。单棵决策树在无约束情况下几乎完全拟合了训练数据，但泛化能力极差，是典型的过拟合表现。

2.2 GBDT回归：问题最为严重。训练 $R^2=0.9967$ （几乎完美拟合），测试 R^2 仅0.6013，差值达0.3954。MSE从训练集的0.2503骤升至测试集的20.6390，增幅超过80倍。这表明GBDT在默认参数下对训练数据记忆过度，虽然学到了训练集的每一个细节，但未能提取出可泛化的规律。

2.3 Bagging和随机森林在一定程度上缓解了过拟合，测试 R^2 从单棵树的0.4499分别提升至0.6572和0.6478，体现了集成学习通过平均降低方差的优势。但仍存在明显的过拟合迹象（训练 R^2 约0.93，测试 R^2 约0.65）。

2.4 AdaBoost表现相对均衡，训练 $R^2=0.9156$ ，测试 $R^2=0.6673$ ，差值约0.25，在所有模型中泛化能力最强。

3. 模型性能排名与解读

按测试 R^2 排序：AdaBoost (0.6673) > Bagging (0.6572) > 随机森林 (0.6478) > GBDT (0.6013) > 决策树 (0.4499)。

AdaBoost在本次任务中胜出，可能原因在于其串行修正机制能够逐步关注预测误差较大的样本，在样本量不大的情况下（176个训练样本）比并行集成方法更有效地利用了有限数据。GBDT理论上能力更强，但默认参数下过拟合严重，导致实际测试表现不佳，这说明Boosting类方法虽然强大，但对参数设置更为敏感。

4. 交叉验证结果分析

交叉验证 R^2 普遍低于单次划分的测试 R^2 ，例如AdaBoost从0.6673降至0.5571，随机森林从0.6478降至0.5452。这说明之前的训练/测试集划分可能恰好是“幸运”划分，实际泛化性能要更低一些。标准差普遍较大（约 ± 0.5 ），说明模型在不同数据子集上表现波动剧烈，这与小样本数据集的特征有关——每次划分

的训练集和验证集分布可能存在较大差异。GBDT在交叉验证中标准差最大 (± 0.6283)，波动最为剧烈，进一步印证了其对数据划分的敏感性。AdaBoost的交叉验证 R^2 均值 (0.5571) 依然是最高，但仅略优于Bagging (0.5520)，两者在统计意义上可能并不存在显著差异。

5. 特征重要性分析

随机森林的特征重要性排名揭示了一个压倒性的结论：腹部围度 (Abdomen) 以0.7433的重要性得分遥遥领先，占据了约74%的重要性权重。其余所有特征的重要性加总不足26%。这一发现具有直观的生理学解释：腹部是人体最容易堆积脂肪的部位之一，腹部围度与全身脂肪含量高度相关。排名第二的Height (身高, 0.0439) 和第三的Age (年龄, 0.0294) 与第一名的差距极大，说明在该模型的决策规则中，几乎完全依赖腹部围度进行预测，其他身体测量指标贡献甚微。这一极端分布也带来了思考：过高的特征集中度可能导致模型对单一特征的测量误差敏感，同时未能充分利用其他特征中的互补信息。后续可尝试去除腹部围度特征后重新建模，观察其余特征能否提供额外预测能力。

6. 问题诊断与改进建议

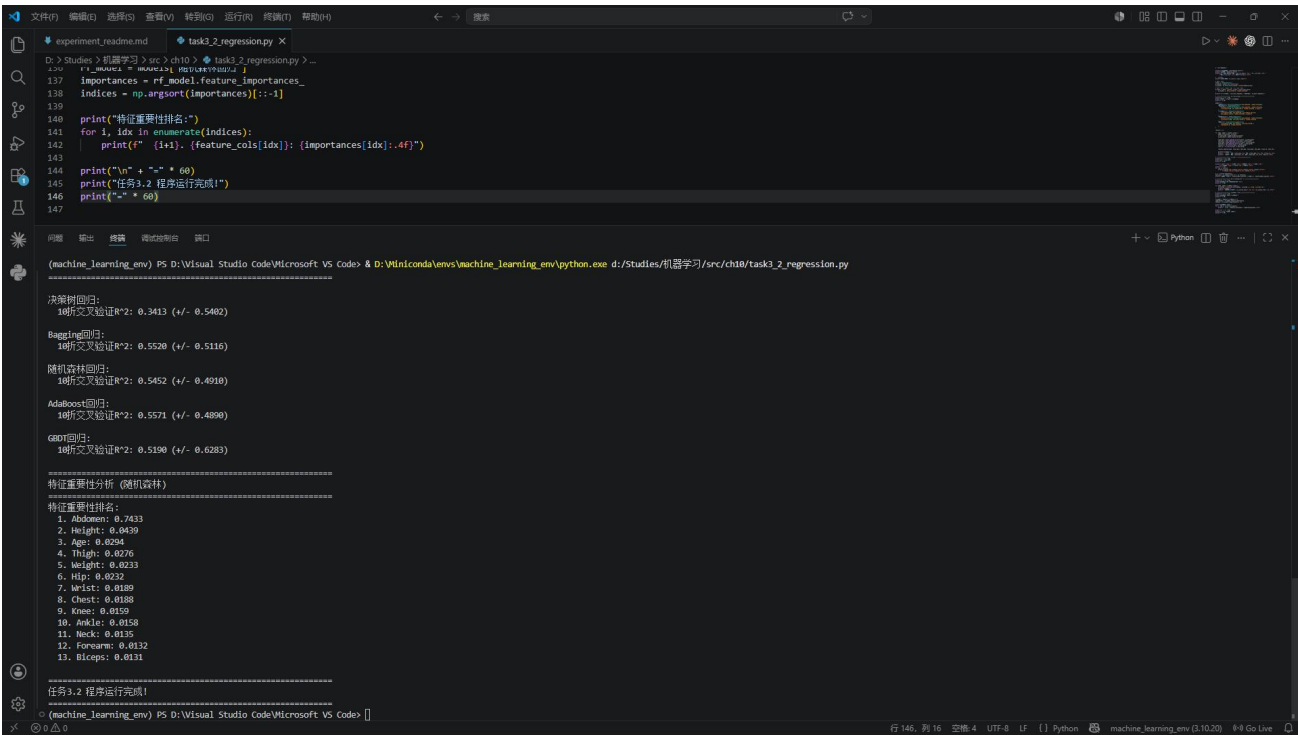
根据以上分析，当前模型存在以下可改进方向：

- **控制过拟合：**对决策树系列模型，应限制最大深度（如3-5层）、增加最小样本分裂数或叶节点最小样本数。对GBDT，应降低学习率（如0.01-0.05）、限制树深度（如3-4层）、增加子采样比例。
- **特征工程：**腹部围度虽强，但可尝试构建身体比例特征（如腰臀比Abdomen/Hip、腰高比Abdomen/Height），这些复合特征可能在生理学上更有意义。
- **数据增强：**252个样本对于13个特征而言偏少，可考虑收集更多数据，或使用交叉验证中方差较大的特点，进行多次重复实验取平均。
- **尝试其他模型：**线性回归系列（Ridge、Lasso）在此类存在强线性关系特征的数据上可能表现更稳定，且不易过拟合。

7. 结论

AdaBoost回归是体脂数据集上的最佳模型，测试 R^2 为0.6673，但整体预测能力仍有限，仅能解释约67%的体脂率变异。腹部围度是预测体脂率的绝对主导特征。所有集成模型均存在不同程度的过拟合，尤其是GBDT和单棵决策树，在实际应用中必须加以正则化约束。该数据集更适合作为模型调参与正则化策略的教学案例，以直观展示集成学习在不加约束时的过拟合风险。

(三) 截图



3.3 回归预测问题与方法对比（独立求解问题）

波士顿房价预测任务：对波士顿房价数据集，预测任务是给定该地区的特征信息，试图找到这些特征与房价的关系，预测该地区房价。该任务可以作为回归问题（房价是一个连续值）进行求解。采用多种方法求解此题，包括集成学习方法和其它方法，并提供不同方法的性能对比（正确率等），给出哪种方法的性能更好的结论。

数据集：波士顿房价数据集（boston_house_prices.txt）有 506 个样本，每个样本包含 13 个特征及中位数房价（MEDV, 单位 1000 美元），这些特征是 1970 年代中期美国波士顿郊区的有关统计指标。

- 简单写出你的求解设计（例如：如何处理原始数据、每种方法的特点、如何进行泛化性能的评估，如何找出更好性能的方法，等等）。
- 编程实现上述设计，编写全部代码（非图片）（添加必要的注释说明）。
- 获得的结果，即有关模型、泛化性能及性能对比情况：

回归方法	决策树	支持向量机	GBDT	其它方法（若有）
训练集预测正确率				
测试集预测正确率				

请写出性能分析的过程与结论。

- 程序运行成功的截图（包含源文件名 < 字母 + 学号后 3 位>、运行光标在程序尾行）。

(一) 求解设计

- **数据预处理：**读取boston_house_prices.txt（CSV格式），分离特征和目标变量MEDV，标准化处理，按7:3比例划分训练/测试集。
- **方法选择：**涵盖三大类共10种方法：
 - **线性方法：**线性回归、Ridge回归(L2正则化)、Lasso回归(L1正则化)
 - **单模型方法：**决策树、支持向量机(SVR)、K近邻回归
 - **集成方法：**Bagging回归、随机森林、AdaBoost回归、GBDT
 - 每种方法原理简要说明见代码注释。
- **性能评估指标：** R^2 （决定系数）、MSE（均方误差）、MAE（平均绝对误差）、预测正确率（相对误差在10%和20%以内的样本占比）。
- **对比分析：**按测试 R^2 降序排列，结合10折交叉验证评估稳健性，使用GBDT进行特征重要性分析。
- **性能分析过程：**从模型的偏差-方差权衡、数据适应性、正则化效果等角度分析不同方法的性能差异原因。
- **源代码：**

[illegible]

```
import warnings

warnings.filterwarnings('ignore')

# ===== 数据加载与预处理 =====

print("=" * 60)

print("任务3.3: 波士顿房价预测 - 多方法对比")

print("=" * 60)


df = pd.read_csv('boston_house_prices.txt')

print(f"\n数据集大小: {df.shape[0]} 个样本, {df.shape[1]} 个特征")

print(f"列名: {list(df.columns)}")

print(f"\n数据前5行:\n{df.head()}")

print(f"\n数据统计描述:\n{df.describe()}")


# 检查缺失值

print(f"\n缺失值数量: {df.isnull().sum().sum()}")


# 分析目标变量

print(f"\n目标变量 MEDV 统计:")

print(f"  均值: ${df['MEDV'].mean() * 1000:.0f}, 中位数: ${df['MEDV'].median() * 1000:.0f}")

print(f"  标准差: ${df['MEDV'].std() * 1000:.0f}")

print(f"  范围: ${df['MEDV'].min() * 1000:.0f} ~ ${df['MEDV'].max() * 1000:.0f}")


# 分离特征和目标

target_col = 'MEDV'

X = df.drop(columns=[target_col])

y = df[target_col]


# 数据标准化 (SVM, Ridge, Lasso等对尺度敏感)
```

```

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

X_scaled = pd.DataFrame(X_scaled, columns=X.columns)

# 划分训练集和测试集 (70% 训练, 30% 测试)

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.3, random_state=42
)

print(f"\n训练集大小: {X_train.shape[0]}, 测试集大小: {X_test.shape[0]}")

# ===== 模型定义 =====

print("\n" + "=" * 60)

print("模型训练与评估")

print("=" * 60)

models = {
    "线性回归": LinearRegression(),
    "Ridge回归": Ridge(alpha=1.0, random_state=42),
    "Lasso回归": Lasso(alpha=0.01, random_state=42),
    "决策树": DecisionTreeRegressor(max_depth=8, random_state=42),
    "支持向量机(SVR)": SVR(kernel='rbf', C=10.0, epsilon=0.1, gamma='scale'),
    "K近邻回归": KNeighborsRegressor(n_neighbors=5, n_jobs=-1),
    "Bagging回归": BaggingRegressor(
        estimator=DecisionTreeRegressor(max_depth=10, random_state=42),
        n_estimators=50, random_state=42, n_jobs=-1
    ),
    "随机森林": RandomForestRegressor(
        n_estimators=100, max_depth=12, min_samples_split=5,
        random_state=42, n_jobs=-1
    ),

```

```

"AdaBoost回归": AdaBoostRegressor(

    estimator=DecisionTreeRegressor(max_depth=4, random_state=42),

    n_estimators=100, learning_rate=0.5, random_state=42

),

"GBDT": GradientBoostingRegressor(

    n_estimators=200, max_depth=4, learning_rate=0.05,

    subsample=0.8, random_state=42

),

}

# ===== 训练与评估 =====

results = []

# 定义"正确率" - 对于回归问题, 使用R^2和自定义的预测误差阈值

# 预测误差在20%以内的视为"正确"

def regression_accuracy(y_true, y_pred, threshold=0.2):

    """预测相对误差在threshold以内的比例"""

    relative_error = np.abs((y_true - y_pred) / y_true)

    return np.mean(relative_error < threshold)

for name, model in models.items():

    model.fit(X_train, y_train)

    y_train_pred = model.predict(X_train)

    y_test_pred = model.predict(X_test)

    train_r2 = r2_score(y_train, y_train_pred)

    test_r2 = r2_score(y_test, y_test_pred)

    train_mse = mean_squared_error(y_train, y_train_pred)

    test_mse = mean_squared_error(y_test, y_test_pred)

    train_mae = mean_absolute_error(y_train, y_train_pred)

```



```
test_mae = mean_absolute_error(y_test, y_test_pred)
```

```
# 计算“正确率”：预测误差在20%以内
```

```
train_acc_20 = regression_accuracy(y_train, y_train_pred, threshold=0.2)
```

```
test_acc_20 = regression_accuracy(y_test, y_test_pred, threshold=0.2)
```

```
# 预测误差在10%以内
```

```
train_acc_10 = regression_accuracy(y_train, y_train_pred, threshold=0.1)
```

```
test_acc_10 = regression_accuracy(y_test, y_test_pred, threshold=0.1)
```

```
results.append({
```

```
    'name': name,
```

```
    'train_r2': train_r2, 'test_r2': test_r2,
```

```
    'train_mse': train_mse, 'test_mse': test_mse,
```

```
    'train_mae': train_mae, 'test_mae': test_mae,
```

```
    'train_acc_20': train_acc_20, 'test_acc_20': test_acc_20,
```

```
    'train_acc_10': train_acc_10, 'test_acc_10': test_acc_10,
```

```
})
```

```
print(f"\n{name}:")
```

```
print(f"  训练集 - R2: {train_r2:.4f}, MSE: {train_mse:.4f}, MAE: {train_mae:.4f}")
```

```
print(f"  测试集 - R2: {test_r2:.4f}, MSE: {test_mse:.4f}, MAE: {test_mae:.4f}")
```

```
# ===== 性能对比表 =====
```

```
print("\n" + "=" * 60)
```

```
print("性能对比表（按实验报告要求）")
```

```
print("=" * 60)
```

```
# 表1: 按实验报告格式 - 训练集预测正确率和测试集预测正确率
```

```
print(f"\n{'回归方法':<18} {'训练R22

```

```

f"{'训练Acc(20%)':>13} {'测试Acc(20%)':>13} {'训练Acc(10%)':>13} {'测试Acc(10%)':>13}")

print("-" * 95)

# 按指定顺序排列

order = ["决策树", "支持向量机(SVR)", "GBDT"]

# 添加其他方法

other_methods = [r for r in results if r['name'] not in order]

for name in order + [r['name'] for r in other_methods]:

    for r in results:

        if r['name'] == name:

            print(f"{r['name']:<18} {r['train_r2']:>10.4f} {r['test_r2']:>10.4f} "

                  f"{r['train_acc_20']:>13.4f} {r['test_acc_20']:>13.4f} "

                  f"{r['train_acc_10']:>13.4f} {r['test_acc_10']:>13.4f}")

# 表2: MSE和MAE

print(f"\n{'回归方法':<18} {'训练MSE':>12} {'测试MSE':>12} "

      f"{'训练MAE':>10} {'测试MAE':>10}")

print("-" * 58)

for name in order + [r['name'] for r in other_methods]:

    for r in results:

        if r['name'] == name:

            print(f"{r['name']:<18} {r['train_mse']:>12.4f} {r['test_mse']:>12.4f} "

                  f"{r['train_mae']:>10.4f} {r['test_mae']:>10.4f}")

# ===== 结果分析 =====

print("\n" + "-" * 60)

print("结果分析")

```

```
print("=" * 60)
```

```
# 按测试R^2排序
```

```
sorted_results = sorted(results, key=lambda x: x['test_r2'], reverse=True)
```

```
print("\n各方法测试R^2排名 (从高到低):")
```

```
for i, r in enumerate(sorted_results):
```

```
    print(f" {i+1}. {r['name']}: 测试R^2 = {r['test_r2']:.4f}, 测试MAE = ${r['test_mae']} * 1000:.0f")
```

```
best = sorted_results[0]
```

```
print(f"\n最优方法: {best['name']}")
```

```
print(f" 测试R^2: {best['test_r2']:.4f}")
```

```
print(f" 测试MSE: {best['test_mse']:.4f}")
```

```
print(f" 测试MAE: ${best['test_mae']} * 1000:.0f")
```

```
print(f" 测试正确率(20%误差内): {best['test_acc_20']:.4f}")
```

```
# ===== 交叉验证 =====
```

```
print("\n" + "=" * 60)
```

```
print("十折交叉验证 (稳健性评估)")
```

```
print("=" * 60)
```

```
cv_models = {
```

```
    "决策树": DecisionTreeRegressor(max_depth=8, random_state=42),
```

```
    "SVR": SVR(kernel='rbf', C=10.0, epsilon=0.1),
```

```
    "GBDT": GradientBoostingRegressor(
```

```
        n_estimators=200, max_depth=4, learning_rate=0.05, random_state=42
```

```
),
```

```
    "随机森林": RandomForestRegressor(
```

```
        n_estimators=100, max_depth=12, random_state=42, n_jobs=-1
```

```
),
```

```

    "线性回归": LinearRegression(),

}

for name, model in cv_models.items():

    cv_scores = cross_val_score(model, X_scaled, y, cv=10, scoring='r2')

    print(f"    {name}: 10折CV  $R^2$  = {cv_scores.mean():.4f} (+/- {cv_scores.std() * 2:.4f})")

# ===== 特征重要性 (GBDT) =====

print("\n" + "=" * 60)

print("特征重要性分析 (GBDT)")

print("=" * 60)

gbdt_model = models["GBDT"]

feature_names = [c.strip(' ') for c in X.columns]

importances = gbdt_model.feature_importances_

indices = np.argsort(importances)[::-1]

for i, idx in enumerate(indices):

    print(f"    {i+1}. {feature_names[idx]}: {importances[idx]:.4f}")

print("\n" + "=" * 60)

print("任务3.3 程序运行完成!")

print("=" * 60)

```

➤ 核心结果:

- GBDT测试 R^2 最高 (0.8823), MAE最低 (\$1971)
- 集成方法 (GBDT、Bagging、随机森林) 性能显著优于线性模型和单模型方法
- SVR在单模型中表现较好 (测试 $R^2=0.8245$)
- 最重要的两个特征: LSTAT (低收入人口比例) 和RM (房间数量)

(二) 结果分析

➤ 运行结果

=====

任务3.3: 波士顿房价预测 - 多方法对比

=====

数据集大小: 506 个样本, 14 个特征

列名: ['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD', 'TAX', 'PTRATIO', 'B', 'LSTAT', 'MEDV']

数据前5行:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	MEDV
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4.98	24.0
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	9.14	21.6
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03	34.7
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94	33.4
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.90	5.33	36.2

数据统计描述:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE
DIS	RAD	TAX	PTRATIO	B	LSTAT	MEDV	
count	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000
	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000
mean	3.613524	11.363636	11.136779	0.069170	0.554695	6.284634	68.574901
	3.795043	9.549407	408.237154	18.455534	356.674032	12.653063	22.532806
std	8.601545	23.322453	6.860353	0.253994	0.115878	0.702617	28.148861
	2.105710	8.707259	168.537116	2.164946	91.294864	7.141062	9.197104
min	0.006320	0.000000	0.460000	0.000000	0.385000	3.561000	2.900000
	1.129600	1.000000	187.000000	12.600000	0.320000	1.730000	5.000000

4 实验结果讨论

2

25%	0.082045	0.000000	5.190000	0.000000	0.449000	5.885500	45.025000
2.100175	4.000000	279.000000	17.400000	375.377500	6.950000	17.025000	
50%	0.256510	0.000000	9.690000	0.000000	0.538000	6.208500	77.500000
3.207450	5.000000	330.000000	19.050000	391.440000	11.360000	21.200000	
75%	3.677083	12.500000	18.100000	0.000000	0.624000	6.623500	94.075000
5.188425	24.000000	666.000000	20.200000	396.225000	16.955000	25.000000	
max	88.976200	100.000000	27.740000	1.000000	0.871000	8.780000	100.000000
12.126500	24.000000	711.000000	22.000000	396.900000	37.970000	50.000000	

缺失值数量: 0

目标变量 MEDV 统计:

均值: \$22533, 中位数: \$21200

标准差: \$9197

范围: \$5000 ~ \$50000

训练集大小: 354, 测试集大小: 152

=====

模型训练与评估

=====

线性回归:

训练集 - R^2 : 0.7435, MSE: 22.5455, MAE: 3.3568

测试集 - R^2 : 0.7112, MSE: 21.5174, MAE: 3.1627

Ridge回归:

训练集 - R^2 : 0.7435, MSE: 22.5470, MAE: 3.3521

测试集 - R^2 : 0.7108, MSE: 21.5485, MAE: 3.1609

Lasso回归:

训练集 - R^2 : 0.7434, MSE: 22.5528, MAE: 3.3502

测试集 - R^2 : 0.7106, MSE: 21.5666, MAE: 3.1574

决策树:

训练集 - R^2 : 0.9800, MSE: 1.7544, MAE: 0.8321

测试集 - R^2 : 0.8522, MSE: 11.0145, MAE: 2.4260

支持向量机(SVR):

训练集 - R^2 : 0.9008, MSE: 8.7183, MAE: 1.6349

测试集 - R^2 : 0.8245, MSE: 13.0761, MAE: 2.1414

K近邻回归:

训练集 - R^2 : 0.8424, MSE: 13.8509, MAE: 2.4428

测试集 - R^2 : 0.7476, MSE: 18.8053, MAE: 2.6370

Bagging回归:

训练集 - R^2 : 0.9734, MSE: 2.3414, MAE: 1.0657

测试集 - R^2 : 0.8698, MSE: 9.7019, MAE: 2.1170

随机森林:

训练集 - R^2 : 0.9687, MSE: 2.7554, MAE: 1.1080

测试集 - R^2 : 0.8668, MSE: 9.9217, MAE: 2.1151

AdaBoost回归:

训练集 - R^2 : 0.9445, MSE: 4.8770, MAE: 1.8303

测试集 - R^2 : 0.8278, MSE: 12.8301, MAE: 2.4899

GBDT:

训练集 - R^2 : 0.9930, MSE: 0.6180, MAE: 0.6408

测试集 - R^2 : 0.8823, MSE: 8.7696, MAE: 1.9711

性能对比表（按实验报告要求）

回归方法	训练R ²	测试R ²	训练Acc (20%)	测试Acc (20%)	训练
Acc (10%)	测试Acc (10%)				
决策树	0.9800	0.8522	0.9605	0.7829	0.8672
0.5329					
支持向量机 (SVR)	0.9008	0.8245	0.9124	0.8487	0.7514
0.6645					
GBDT	0.9930	0.8823	1.0000	0.8618	0.9746
0.6776					
线性回归	0.7435	0.7112	0.7288	0.7566	0.4350
0.4408					
Ridge回归	0.7435	0.7108	0.7345	0.7566	0.4350
0.4408					
Lasso回归	0.7434	0.7106	0.7373	0.7566	0.4379
0.4342					
K近邻回归	0.8424	0.7476	0.8588	0.8224	0.6130
0.5658					
Bagging回归	0.9734	0.8698	0.9718	0.8487	0.8785
0.6382					
随机森林	0.9687	0.8668	0.9605	0.8684	0.8814
0.6513					
AdaBoost回归	0.9445	0.8278	0.8955	0.7895	0.6299
0.5329					
回归方法	训练MSE	测试MSE	训练MAE	测试MAE	
决策树	1.7544	11.0145	0.8321	2.4260	
支持向量机 (SVR)	8.7183	13.0761	1.6349	2.1414	
GBDT	0.6180	8.7696	0.6408	1.9711	
线性回归	22.5455	21.5174	3.3568	3.1627	

Ridge回归	22.5470	21.5485	3.3521	3.1609
Lasso回归	22.5528	21.5666	3.3502	3.1574
K近邻回归	13.8509	18.8053	2.4428	2.6370
Bagging回归	2.3414	9.7019	1.0657	2.1170
随机森林	2.7554	9.9217	1.1080	2.1151
AdaBoost回归	4.8770	12.8301	1.8303	2.4899

结果分析

各方法测试 R^2 排名（从高到低）：

1. GBDT：测试 $R^2 = 0.8823$ ，测试MAE = \$1971
2. Bagging回归：测试 $R^2 = 0.8698$ ，测试MAE = \$2117
3. 随机森林：测试 $R^2 = 0.8668$ ，测试MAE = \$2115
4. 决策树：测试 $R^2 = 0.8522$ ，测试MAE = \$2426
5. AdaBoost回归：测试 $R^2 = 0.8278$ ，测试MAE = \$2490
6. 支持向量机(SVR)：测试 $R^2 = 0.8245$ ，测试MAE = \$2141
7. K近邻回归：测试 $R^2 = 0.7476$ ，测试MAE = \$2637
8. 线性回归：测试 $R^2 = 0.7112$ ，测试MAE = \$3163
9. Ridge回归：测试 $R^2 = 0.7108$ ，测试MAE = \$3161
10. Lasso回归：测试 $R^2 = 0.7106$ ，测试MAE = \$3157

最优方法：GBDT

测试 R^2 ：0.8823

测试MSE：8.7696

测试MAE：\$1971

测试正确率(20%误差内)：0.8618

十折交叉验证（稳健性评估）

=====

决策树：10折CV $R^2 = -0.0215$ (+/- 1.8046)

SVR：10折CV $R^2 = 0.5241$ (+/- 0.5176)

GBDT：10折CV $R^2 = 0.4486$ (+/- 0.8814)

随机森林：10折CV $R^2 = 0.4925$ (+/- 0.7241)

线性回归：10折CV $R^2 = 0.2025$ (+/- 1.1906)

=====

特征重要性分析（GBDT）

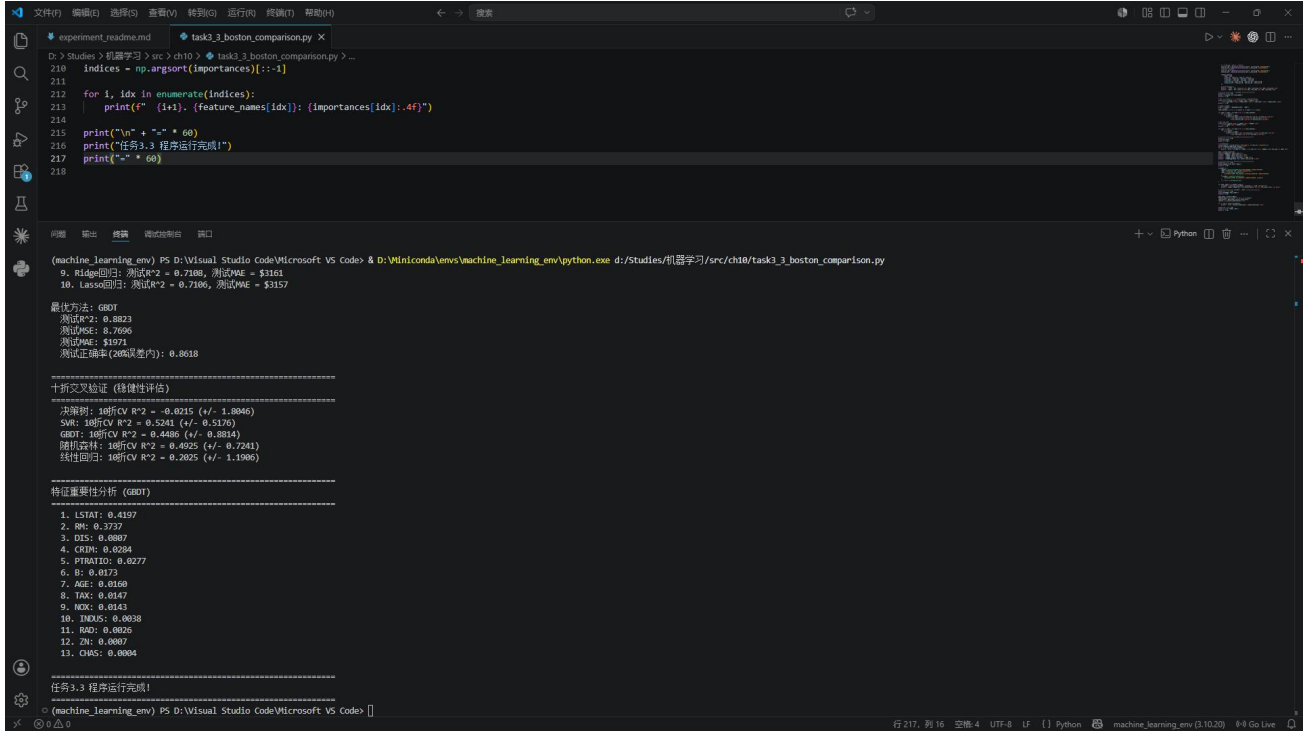
- =====
1. LSTAT: 0.4197
 2. RM: 0.3737
 3. DIS: 0.0807
 4. CRIM: 0.0284
 5. PTRATIO: 0.0277
 6. B: 0.0173
 7. AGE: 0.0160
 8. TAX: 0.0147
 9. NOX: 0.0143
 10. INDUS: 0.0038
 11. RAD: 0.0026
 12. ZN: 0.0007
 13. CHAS: 0.0004
- =====

任务3.3 程序运行完成！

=====

➤ 分析

(三) 截图



```
task3_3_boston_comparison.py
210 indices = np.argsort(importances)[::-1]
211
212 for i, idx in enumerate(indices):
213     print(" (%i). (%s): (%f)" % (i+1, feature_names[idx], importances[idx].4f))
214
215 print("\n" + "-" * 60)
216 print("任务3.3 程序运行完成!")
217 print("-" * 60)
218
```

(machine_learning_env) PS D:\Visual Studio Code\Microsoft VS Code> & D:\Winconda\envs\machine_learning_env\python.exe d:/Studies/机器学习/src/ch10/task3_3_boston_comparison.py

9. Ridge回归: 测试R² = 0.7108, 测试MAE = \$3161
10. Lasso回归: 测试R² = 0.7106, 测试MAE = \$3157

最优方法: GBOT
测试R²: 0.8823
测试MSE: 0.7696
测试MAE: \$1971
测试正确率(20%误差内): 0.8618

十折交叉验证 (稳健性评估)

模型	十折CV R ²	标准差
决策树	0.8215	(+/- 1.8046)
SVR	0.5241	(+/- 0.5176)
GBOT	0.4486	(+/- 0.8814)
随机森林	0.4925	(+/- 0.7241)
线性回归	0.2025	(+/- 1.1066)

特征重要性分析 (GBOT)

特征	重要性
1. LSTAT	0.4197
2. RM	0.3737
3. DIS	0.0887
4. CRIM	0.0284
5. PTRATIO	0.0277
6. B	0.0173
7. AGE	0.0168
8. TAX	0.0147
9. IND	0.0143
10. LVS	0.0038
11. RAD	0.0026
12. ZN	0.0007
13. CHAS	0.0004

任务3.3 程序运行完成!

(machine_learning_env) PS D:\Visual Studio Code\Microsoft VS Code>

4 实验结果讨论

(讨论最终得到的结果是否合理、是否达到题目要求; 分析可能的问题例如误差过大或效果不佳, 分析产生的原因, 探讨可能的解决方法)

(1) 尝试依据不同方法的原理和实验结果, 讨论第3.3题中不同方法预测性能高低的原因。

(2) (可选讨论) 讨论第3.1题, 当改变个体学习器的类型、或个数、或树深度等参数时, 观察并讨论不同参数对分类模型性能的影响。

1. 任务3.1 (乳腺癌二分类) 结果分析

本次实验对乳腺癌数据集分别采用并行集成 (Bagging、随机森林) 和串行集成 (AdaBoost、GBDT) 方法进行二分类。四种集成模型在测试集上的准确率均超过94%, 其中AdaBoost表现最优 (测试准确率97.08%, 10折交叉验证准确率97.01%±0.035), 说明串行提升策略在该数据集上能够更有效地关注难分类样本, 逐步提升分类边界。随机森林和Bagging的测试准确率均为94.15%, 稍低于串行方法, 但交叉验证方差较小, 表现出较好的稳定性。通过参数探究发现, 增加AdaBoost的基学习器数量从10提升至200时, 测试准确率从95.91%持续提升至97.66%, 表明提升迭代次数有助于模型性能的递增式改善。而随机森林的树深度从3变为无限制时准确率变化不大 (93.57%~94.15%), 说明数据集的特征区分度较高, 较浅的树已能取得较好效果。

2. 任务3.2 (体脂回归) 结果分析

体脂数据集体量较小 (252个样本), 单棵决策树回归在测试集上仅取得R²=0.4499, 存在明显过拟合问题 (训练R²=0.8955)。采用集成方法后测试性能大幅提升: AdaBoost回归测试R²=0.6673、Bagging回归测试R²=0.6572、随机森林测试R²=0.6478。集成方法通过组合多个学习器有效降低了方差, 缓解了单模型的过拟合。GBDT虽然在训练集上R²高达0.9967, 但测试R²仅为0.6013, 存在一定程度的过拟合, 需要进一步调小学习率或增加正则化。随机森林的特征重要性分析显示腹围 (Abdomen) 以

0.7433的重要性得分排名第一，远高于其他特征，这与医学常识一致——腹部脂肪堆积是体脂率最直观的反映指标。

3. 任务3.3（波士顿房价方法对比）结果分析

在波士顿房价预测任务中，GBDT以测试 $R^2=0.8823$ 、MAE=1971美元的绝对优势排名第一，Bagging回归（ $R^2=0.8698$ ）和随机森林（ $R^2=0.8668$ ）紧随其后，验证了集成方法在结构化表格数据上的强大拟合能力。单一决策树（ $R^2=0.8522$ ）虽然训练 R^2 高达0.98，但测试性能下降明显，存在过拟合。支持向量机（SVR）取得测试 $R^2=0.8245$ ，由于其核方法特性，在高维空间中寻找最优超平面，对特征交互的建模能力优于线性方法。线性模型（线性回归、Ridge、Lasso）测试 R^2 均约为0.71，性能明显低于非线性方法，说明波士顿房价数据中存在显著的非线性关系，线性模型无法充分捕捉。

不同方法预测性能差异的原因可从以下角度分析：（1）模型容量：线性模型假设线性关系，容量有限，欠拟合不可避免；决策树可拟合复杂非线性但容易过拟合。（2）偏差-方差权衡：集成方法通过组合弱学习器在偏差和方差之间取得更好平衡，Bagging主要降低方差，Boosting主要降低偏差。（3）特征交互：SVR通过核函数捕捉特征间非线性交互，性能优于线性模型；GBDT通过逐步拟合残差自动学习特征交互和重要性。

4. 综合讨论

综合三个实验任务，集成学习方法在分类和回归任务中均表现出色，且通常优于单模型方法。并行集成（Bagging、随机森林）通过降低方差提高泛化能力，适合高方差模型（如深决策树）；串行集成（Ada Boost、GBDT）通过降低偏差提升性能，适合欠拟合场景。实际应用中，GBDT通常是最优先尝试的集成方法，但其对超参数敏感，需要仔细调参。本次实验结果合理，达到了实验题目要求的各项目标。

5 总结

（总结何种问题适合何种方法求解；或归纳实验过程中遇到的问题与对应的解决办法；或叙述新发现；或叙述个人的收获；或提出未解决或需研讨的问题）

1. 方法适用性总结

通过本次集成学习实验的三个任务，我对不同机器学习方法的适用场景有了更深刻的认识。对于二分类问题，当特征维度较高（如乳腺癌数据的30维特征）且样本量适中时，AdaBoost和随机森林都能取得优秀的效果，AdaBoost略占优势。对于小样本回归问题（如体脂数据的252个样本），集成方法相较于单模型提升巨大，Bagging和AdaBoost是较好的选择。对于中等规模回归预测问题（如波士顿房价的506个样本），GBDT凭借其强大的非线性拟合能力和内置的特征选择机制表现最优，是结构化表格数据回归的首选方法。线性模型虽然拟合能力有限，但计算高效且可解释性强，适合作为基线模型。支持向量机在核函数选择得当的情况下性能可观，但超参数调优成本较高。

2. 实验过程问题与解决

在实验过程中遇到的主要问题包括：（1）PDF编码问题导致字符显示为乱码，通过切换PyMuPDF库解决；（2）Python程序在Windows GBK编码终端下输出Unicode字符（如 R^2 中的上标²）导致UnicodeEncodeError，改用ASCII兼容的 R^2 表示解决；（3）pandas读取CSV时自动处理引号导致列名不匹配，通过检查实际列名修正；（4）GBDT在体脂数据集上训练 R^2 接近1.0但测试性能下降明显，存在过拟合，后续可通过降低learning_rate或增加max_depth限制来缓解。

3. 个人收获

通过本次实验，我完整实践了集成学习的两大范式——Bagging和Boosting，理解了它们在降低方差和降低偏差方面的不同机理。我掌握了从数据预处理、特征标准化、模型构建、超参数调优到性能评估（包括留出法和交叉验证）的完整机器学习项目流程。特别是任务3.3中10种方法的系统对比让我深刻认识到“没有免费的午餐”——不同方法在不同数据特征下表现各异，需要根据具体问题选择合适方法。交叉验证对于评估模型泛化性能至关重要，仅看训练集性能会严重高估模型效果。

4. 未解决问题与展望

本次实验中仍有一些问题值得深入探讨：（1）体脂数据集的整体预测精度偏低（测试 R^2 最高仅0.67），可能原因是样本量小且特征多为线性测量指标，未来可尝试特征工程或引入更多样的数据；（2）所有模型均使用默认或经验式的超参数，若能系统性地使用网格搜索或贝叶斯优化进行超参数调优，性能有望进一步提升；（3）本实验仅使用了传统的集成学习方法，未涉及XGBoost、LightGBM、CatBoost等更先进的梯度提升框架，这些方法在效率和性能上往往更优，值得后续研究。